

Database Query Optimization & Cost Analysis Report

1. Finding the Second Highest Salary

- Sub-optimal (Nested Subquery):

```
EXPLAIN ANALYZE
SELECT MAX(emp.salary)
FROM emp
WHERE emp.salary < (SELECT MAX(emp.salary) FROM emp);
```

- > Aggregate: max(emp.salary) (cost=3.85 rows=1) (actual time=0.3..0.3 rows=1 loops=1)
- > Filter: (emp.salary < (select #2)) (cost=2.05 rows=18) (actual time=0.248..0.292 rows=52 loops=1)
- > Table scan on emp (cost=2.05 rows=54) (actual time=0.163..0.201 rows=54 loops=1)
- > Select #2 (subquery in condition; run only once)
- > Aggregate: max(emp.salary) (cost=11.1 rows=1) (actual time=0.0694..0.0695 rows=1 loops=1)
- > Table scan on emp (cost=5.65 rows=54) (actual time=0.0048..0.0572 rows=54 loops=1)

- **Observation:** The database performs a **Table Scan twice**. It scans once to find the global maximum, then scans again to filter it out and find the next highest value.
- **Finding:** This is inefficient because it "re-reads" the same data. The cost will grow significantly as the table size increases.

- Optimized (Order By with Offset):

```
EXPLAIN ANALYZE
SELECT DISTINCT emp.salary
FROM emp
ORDER BY emp.salary DESC
LIMIT 1 OFFSET 1;
```

-> Limit/Offset: 1/1 row(s) (actual time=0.154..0.154 rows=1 loops=1)
-> Sort: emp.empsalary DESC, limit input to 2 row(s) per chunk (actual time=0.152..0.152 rows=2 loops=1)
-> Table scan on <temporary> (cost=11.1..14.2 rows=54) (actual time=0.12..0.124 rows=29 loops=1)
-> Temporary table with deduplication (cost=11.1..11.1 rows=54) (actual time=0.118..0.118 rows=29 loops=1)
-> Table scan on emp (cost=5.65 rows=54) (actual time=0.0302..0.0753 rows=54 loops=1)

- **Observation:** The plan shows only **one Table Scan**. The engine collects the unique salaries, sorts them, and immediately grabs the second row.
- **Finding:** This query is **roughly 50% faster** than the first approach. It is better because it only processes the data once.

2. Comparing IN vs EXISTS

EXPLAIN ANALYZE

SELECT customer_id

FROM customer

WHERE customer_id IN(SELECT customer_id FROM order_header);

-> Nested loop inner join (cost=116979 rows=1.17e+6) (actual time=1.22..2.38 rows=633 loops=1)

-> Covering index scan on customer using PRIMARY (cost=80.2 rows=792) (actual time=0.0735..0.42 rows=792 loops=1)

-> Single-row index lookup on <subquery2> using <auto_distinct_key> (customer_id=customer.customer_id) (cost=297..297 rows=1) (actual time=0.00205..0.00218 rows=0.799 loops=792)

-> Materialize with deduplication (cost=296..296 rows=1475) (actual time=1.14..1.14 rows=633 loops=1)

-> Covering index scan on order_header using customer_id (cost=149 rows=1475) (actual time=0.0295..0.529 rows=1475 loops=1)

EXPLAIN ANALYZE

SELECT customer_id

FROM customer c

WHERE EXISTS (SELECT 1 FROM order_header o WHERE c.customer_id = o.customer_id);

-> Nested loop inner join (cost=116979 rows=1.17e+6) (actual time=0.933..1.71 rows=633 loops=1)

-> Covering index scan on c using PRIMARY (cost=80.2 rows=792) (actual time=0.0778..0.307 rows=792 loops=1)

-> Single-row index lookup on <subquery2> using <auto_distinct_key>

(customer_id=c.customer_id) (cost=297..297 rows=1) (actual time=0.00151..0.00159 rows=0.799 loops=792)

-> Materialize with deduplication (cost=296..296 rows=1475) (actual time=0.845..0.845 rows=633 loops=1)

-> Covering index scan on o using customer_id (cost=149 rows=1475) (actual time=0.0814..0.476 rows=1475 loops=1)

Observation:

Queries: IN Subquery and EXISTS Correlated Subquery.

Both queries resulted in an **identical Cost of 116,979**. This reveals how the modern MySQL optimizer handles these operations.

1. Optimizer Rewriting (Semijoin): Even though I wrote the queries differently, the database engine recognized they were doing the same job. It automatically "rewrote" both of them into a **Semijoin**. A Semijoin is very efficient because it stops looking in the order_header table the moment it finds the *first* match for a customer.

2. Covering Indexes: Both queries used a **Covering Index Scan**. This means the database didn't have to look at the actual table data on the disk; it found everything it needed inside the Index (RAM), which is the fastest way to retrieve data.

3. The Difference: Historically, IN was considered slower because older databases would sometimes try to build the entire subquery list in memory first. However, my analysis shows that in modern versions, the performance is exactly the same for indexed columns.

3. Filtering by Date

- Sub-optimal (Function on Column):

```
EXPLAIN ANALYZE
SELECT order_id
FROM order_header
WHERE MONTH(order_date) = 3;
```

-> Filter: (month(order_header.order_date) = 3) (cost=149 rows=1475) (actual time=0.18..0.591 rows=675 loops=1)

-> Covering index scan on order_header using idx_order_date (cost=149 rows=1475) (actual time=0.092..0.398 rows=1475 loops=1)

- **Finding:** Wrapping the column in a function prevents index usage. The database must calculate MONTH() for **every single row** (1,475 rows). It is a "brute-force" approach.

- Optimized (Range Comparison):

```
EXPLAIN ANALYZE SELECT order_id
FROM order_header
WHERE order_date >= '2026-03-01' AND order_date < '2026-03-31';
```

-> Filter: ((order_header.order_date >= DATE'2026-03-01') and (order_header.order_date < DATE'2026-03-31')) (cost=131 rows=650) (actual time=0.0202..0.3 rows=650 loops=1)

-> Covering index range scan on order_header using idx_order_date over ('2026-03-01' <= order_date < '2026-03-31') (cost=131 rows=650) (actual time=0.0174..0.207 rows=650 loops=1)

The column is "naked" (no functions applied). This allows the database to use an index to "jump" straight to March 1st and stop as soon as it hits April 1st.

- **Finding:** This query is significantly more efficient. Instead of calculating a function for every row, it performs a targeted search.

- **Key Observation:** The plan shows a **Covering index range scan**. Instead of looking at 1,475 rows, it only looked at **650 rows**.

4. Subquery vs JOIN: Finding all customers who have placed an order after April 1st, 2026

- **Sub-optimal (IN Clause with Filter):**

```
EXPLAIN ANALYZE
SELECT customer_id
FROM customer
WHERE customer_id IN(SELECT customer_id FROM order_header WHERE order_date >
'2026-04-01');
```

-> Nested loop inner join (cost=61539 rows=613800) (actual time=0.758..1.37 rows=472 loops=1)

-> Covering index scan on customer using PRIMARY (cost=80.2 rows=792) (actual time=0.046..0.252 rows=792 loops=1)

-> Single-row index lookup on <subquery2> using <auto_distinct_key> (customer_id=customer.customer_id) (cost=227..227 rows=1) (actual time=0.00123..0.00128 rows=0.596 loops=792)

-> Materialize with deduplication (cost=226..226 rows=775) (actual time=0.707..0.707 rows=472 loops=1)

-> Filter: (order_header.order_date > DATE'2026-04-01') (cost=149 rows=775) (actual time=0.0277..0.493 rows=775 loops=1)

-> Table scan on order_header (cost=149 rows=1475) (actual time=0.0199..0.379 rows=1475 loops=1)

Total Cost: 61,539

- **Key Observation:** The database performs a process called "**Materialize with deduplication.**" It essentially has to build a temporary internal table for the subquery (<subquery2>) before it can even start matching it with the customer table.
- **The Problem:** Because it has to look up every customer against this materialized subquery 792 times (loops=792), the cost explodes.

- Optimized (INNER JOIN with DISTINCT):

```
EXPLAIN ANALYZE
SELECT DISTINCT customer_id
FROM customer
JOIN order_header USING(customer_id)
WHERE order_date > '2026-04-01';
```

-> Table scan on <temporary> (cost=498..510 rows=775) (actual time=1.56..1.61 rows=472 loops=1)

-> Temporary table with deduplication (cost=498..498 rows=775) (actual time=1.56..1.56 rows=472 loops=1)

-> Nested loop inner join (cost=420 rows=775) (actual time=0.0733..1.39 rows=775 loops=1)

 -> Filter: (order_header.order_date > DATE'2026-04-01') (cost=149 rows=775) (actual time=0.0591..0.62 rows=775 loops=1)

-> Table scan on order_header (cost=149 rows=1475) (actual time=0.0497..0.477 rows=1475 loops=1)

-> Single-row covering index lookup on customer using PRIMARY

(customer_id=order_header.customer_id) (cost=0.25 rows=1) (actual time=799e-6..827e-6 rows=1 loops=775)

- **Total Cost: 498** (to 510)
- **Key Observation:** The engine filters the order_header table first and then uses a **Single-row covering index lookup** on the customer table's Primary Key. This is much more direct and uses fewer system resources.

5. Implicit vs. Explicit Joins

```
EXPLAIN ANALYZE
SELECT *
FROM customer c, order_header o
WHERE c.customer_id = o.customer_id;
```

-> Nested loop inner join (cost=665 rows=1475) (actual time=0.0634..2.16 rows=1475 loops=1)
-> Table scan on o (cost=149 rows=1475) (actual time=0.0481..0.574 rows=1475 loops=1)
-> Single-row index lookup on c using PRIMARY (customer_id=o.customer_id) (cost=0.25 rows=1) (actual time=890e-6..916e-6 rows=1 loops=1475)

```
EXPLAIN ANALYZE
SELECT *
FROM customer c
JOIN order_header o
ON c.customer_id = o.customer_id;
```

-> Nested loop inner join (cost=665 rows=1475) (actual time=0.119..2.28 rows=1475 loops=1)
-> Table scan on o (cost=149 rows=1475) (actual time=0.104..0.65 rows=1475 loops=1)
-> Single-row index lookup on c using PRIMARY (customer_id=o.customer_id) (cost=0.25 rows=1) (actual time=917e-6..946e-6 rows=1 loops=1475)

Optimizer Intelligence: The MySQL engine is smart enough to "rewrite" the older comma syntax into a standard inner join internally. This is why the execution steps are exactly the same.

Identical Execution: Both queries performed a Single-row index lookup on the customer table 1,475 times. Since they use the same indexes and the same path, there is **zero performance difference**.

Readability vs. Risk: Even though the speed is the same, the **Implicit (Comma)** syntax is much riskier. If you forget the WHERE clause, the database will attempt to match every customer with every order (a Cross Join), which could crash a production system.